



Computer Networks

Week 6 — Detailed Study Notes

Detecting & Reacting to Loss: Duplicate ACKs, Fast Retransmit, Fast Recovery · AIMD ·
TCP BBR · Flow Control meets Congestion Control

IIT Madras BS — Prof. Ayon Chakraborty

How to read these notes. Every slide from the three Week-6 lectures appears here, with the professor's figures embedded. **Module order follows Seek**, which is also the logical order:

- Module 6.1 = Seek L1 (your file “Week-6_Lecture-2”): detecting and reacting to packet loss.
- Module 6.2 = Seek L2 (file “Week-6_Lecture-3”): AIMD and the loss-reaction summary.
- Module 6.3 = Seek L3 (file “Week-6_Lecture-1”): TCP BBR, getting to steady state, flow vs congestion control.

Box types as in earlier weeks. The end has a formula sheet, numerical problems with full solutions, and MCQs with explanations.

Contents

1	Module 6.1 — Detecting and Reacting to Packet Loss	2
1.1	Detecting packet loss: the problem with RTO	2
1.2	Can we detect loss earlier than RTO? Duplicate ACKs	2
1.3	Fast retransmit & fast recovery	3
1.3.1	Distinction: in-flight versus window	3
1.4	TCP fast retransmit (RFC 2581)	4
1.5	TCP fast recovery (RFC 2581)	6
1.6	The full AIMD picture	7
2	Module 6.2 — AIMD and the Loss-Reaction Summary	8
2.1	Why multiplicative decrease gives fairness	9
2.2	Summary: TCP loss detection & reaction	9
3	Module 6.3 — TCP BBR and Getting to Steady State	10
3.1	The zoo, and two algorithms compared	10
3.2	BBR: finding the bottleneck link rate	11
3.3	BBR: adjustments by gain cycling	12
3.4	Summary: getting to steady state	13
3.5	Interaction between flow control and congestion control	13
4	Formula Sheet	14
5	Numerical Problems with Worked Solutions	14
6	Multiple-Choice Questions with Explanations	16

1 Module 6.1 — Detecting and Reacting to Packet Loss

1.1 Detecting packet loss: the problem with RTO

From the lecture slides: Seek W6 L1, slides 2–3 — RTO is coarse; can we detect loss earlier using duplicate ACKs?

Detecting packet loss

- So far, all the algorithms we've studied have a coarse loss detection mechanism: RTO timer expiration
 - Let the RTO expire, drop `cwnd` all the way to 1 MSS
- Analogy: you're driving a car
 - You're waiting until the next car in front is super close to you (RTO) and then hitting the brakes really hard (set `cwnd := 1`)
 - Q: Can you see obstacles from afar and slow down proportionately?
- That is, can the sender see packet loss coming in advance?
 - And reduce `cwnd` more gently?

Can we detect loss earlier than RTO?

- Key idea: use the information in the ACKs. **How?**
- Suppose successive (cumulative) ACKs contain the same ACK#
 - Also called **duplicate ACKs**
 - Occur when network is reordering packets, or one (but not most) packets in the window were lost
- Reduce `cwnd` when you see many duplicate ACKs
 - Consider many dup ACKs a strong indication that packet was lost
 - Default threshold: 3 dup ACKs, i.e., **triple duplicate ACK**
 - **Make `cwnd` reduction gentler than setting `cwnd = 1`; recover faster**

What the slide says (Seek W6 L1, slide 2)

So far, all the algorithms we've studied have a **coarse loss detection** mechanism: RTO timer expiration — let the RTO expire, drop `cwnd` all the way to 1 MSS. Analogy: you're driving a car, waiting until the car in front is super close to you (RTO) and then hitting the brakes really hard (set `cwnd := 1`). Q: can you see obstacles from afar and slow down proportionately? That is, can the sender see packet loss coming in advance, and reduce `cwnd` more gently?

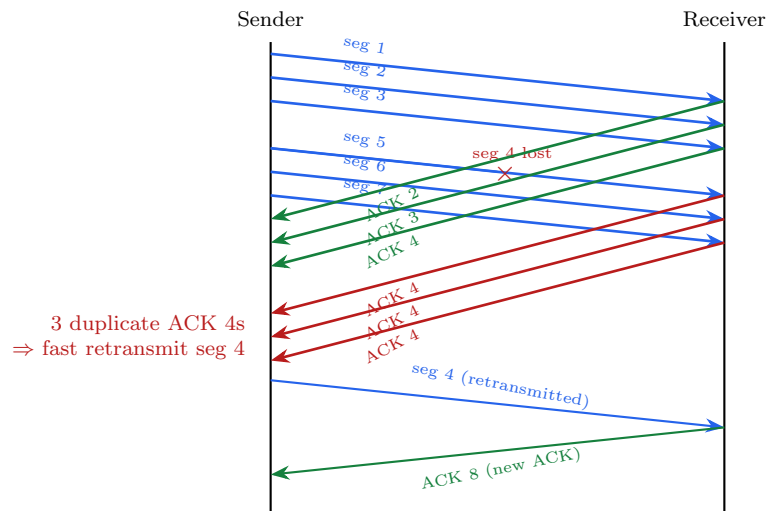
Intuition: Two costs of relying only on RTO

(1) **Waiting:** the RTO is set conservatively ($\text{EstRTT} + 4 \times \text{DevRTT}$, often ≥ 200 ms), so the sender idles for a long time after a single loss. (2) **Over-reacting:** dropping `cwnd` to 1 MSS throws away everything learnt about the path, even when only one packet out of a large window was lost and the rest got through fine.

1.2 Can we detect loss earlier than RTO? Duplicate ACKs

What the slide says (Seek W6 L1, slide 3)

Key idea: use the information in the ACKs. Suppose successive (cumulative) ACKs contain the **same ACK#** — also called **duplicate ACKs**. They occur when the network is reordering packets, or one (but not most) packets in the window were lost. Reduce `cwnd` when you see many duplicate ACKs: consider many dup ACKs a strong indication that a packet was lost. Default threshold: **3 dup ACKs**, i.e. **triple duplicate ACK**. Make the `cwnd` reduction gentler than setting `cwnd = 1`; recover faster.



Intuition: Why “triple”, not one?

One or two duplicate ACKs can be caused by mere **reordering**: segment 4 took a slightly slower path and arrives just after 5. Three duplicates means three later segments overtook segment 4 — much more likely a genuine loss. Three is an engineering compromise: lower and you retransmit needlessly on reordering; higher and you wait too long. And notice the good news hidden in dup ACKs: each one means *a later segment reached the receiver* — the path is still working, so there’s no need to collapse cwnd to 1.

Common pitfall: Dup-ACK numbering

With TCP’s “next expected” ACKs, when segment 4 is lost, segments 5, 6, 7 each produce “ACK 4”. The *first* ACK 4 (after segment 3) is not a duplicate; the three triggered by 5, 6, 7 are the duplicates.

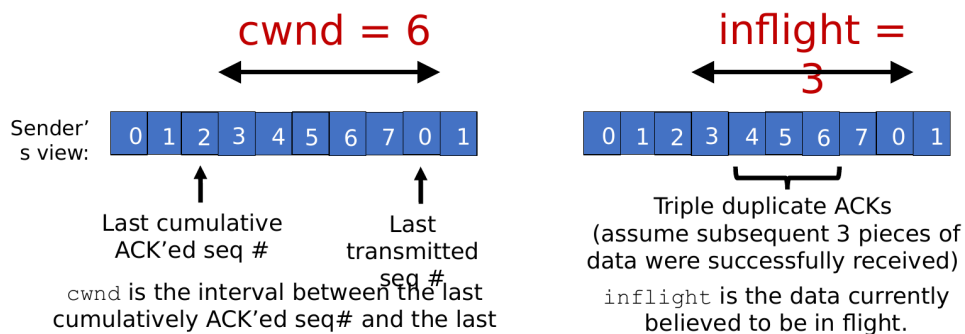
1.3 Fast retransmit & fast recovery

1.3.1 Distinction: in-flight versus window

From the lecture slides: Seek W6 L1, slide 5 — cwnd = 6 vs inflight = 3 after triple duplicate ACKs

Distinction: In-flight versus window

- So far, window and in-flight referred to the same data
- Fast retransmit & fast recovery differentiate the two notions



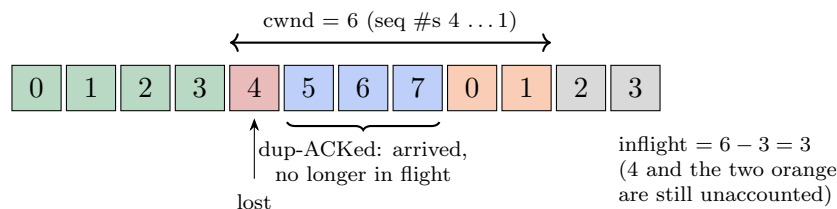
What the slide says (Seek W6 L1, slide 5)

So far, window and in-flight referred to the same data. Fast retransmit & fast recovery differentiate the two notions. **cwnd** is the interval between the last cumulatively ACKed seq# and the last transmitted seq#. **inflight** is the data currently believed to be in flight. Triple duplicate ACKs: assume the subsequent 3 pieces of data were successfully received (so they are no longer in flight).

Definition: cwnd vs inflight

- **cwnd** (window): from the last cumulatively ACKed byte to the last byte the sender may send — a range of *sequence numbers*.
- **inflight**: the amount of data the sender believes is *actually still in the network*. Each duplicate ACK tells it that one more segment has *left* the network (it reached the receiver), even though the cumulative ACK hasn't moved.

Normally $\text{inflight} = \text{cwnd}$. During loss recovery, $\text{inflight} < \text{cwnd}$.



1.4 TCP fast retransmit (RFC 2581)

From the lecture slides: Seek W6 L1, slides 6–7 — the two actions of fast retransmit; multiplicative decrease

TCP fast retransmit (RFC 2581)

- The fact that ACKs are coming means that data is getting delivered to the receiver, albeit with some loss.
- Note: Before the dup ACKs arrive, we assume $\text{inflight} = \text{cwnd}$
- TCP sender does two actions with fast retransmit

TCP fast retransmit (RFC 2581)

- (1) Reduce the **cwnd** and **in-flight** gently
 - Don't drop **cwnd** all the way down to 1 MSS
- Reduce the amount of in-flight data **multiplicatively**
 - Set $\text{inflight} \rightarrow \text{inflight} / 2$
 - That is, set $\text{cwnd} = (\text{inflight} / 2) + 3\text{MSS}$
 - This step is called **multiplicative decrease**
 - Algorithm also sets **ssthresh** to $\text{inflight} / 2$

What the slide says (Seek W6 L1, slides 6–7)

The fact that ACKs are coming means that data is getting delivered to the receiver, albeit with some loss. Before the dup ACKs arrive, we assume $\text{inflight} = \text{cwnd}$. The TCP sender does two actions with fast retransmit. **(1) Reduce cwnd and in-flight gently:** don't drop **cwnd** all the way down to 1 MSS. Reduce the amount of in-flight data **multiplicatively:** set $\text{inflight} \rightarrow \text{inflight}/2$, i.e. set $\text{cwnd} = (\text{inflight}/2) + 3 \text{ MSS}$. This step is called **multiplicative decrease**. The algorithm also sets **ssthresh** to $\text{inflight}/2$.

Formula: Fast retransmit on the 3rd duplicate ACK

Let F = inflight just before the triple dup ACK (= cwnd). Then

$$\begin{aligned} \text{ssthresh} &\leftarrow F/2 \\ \text{inflight target} &\leftarrow F/2 \\ \text{cwnd} &\leftarrow F/2 + 3 \text{ MSS} \\ &\text{retransmit the missing segment immediately} \end{aligned}$$

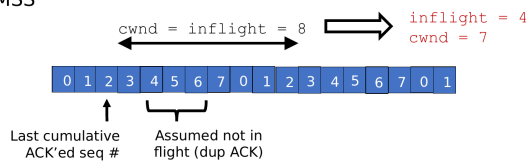
Why +3 MSS? cwnd measures the *sequence range* from the last cumulative ACK. Three segments inside that range are known to have left the network (they caused the 3 dup ACKs) but can't be cumulatively ACKed yet because of the hole. Adding 3 MSS "inflates" the range so that the amount *actually* in flight is $F/2$:

$$\text{inflight} = \text{cwnd} - 3 = F/2.$$

From the lecture slides: Seek W6 L1, slides 8–9 — example: cwnd = inflight = 8 $\Rightarrow$ inflight = 4, cwnd = 7; retransmit immediately

TCP fast retransmit (RFC 2581)

- Example: Suppose cwnd and inflight (before triple dup ACK) were both 8 MSS.
- After triple dup ACK, reduce inflight to 4 MSS
- Assume 3 of those 8 MSS no longer in flight; set cwnd = 7 MSS

**TCP fast retransmit (RFC 2581)**

- (2) The seq# from dup ACKs is **immediately retransmitted**
- That is, **don't wait for an RTO** if there is sufficiently strong evidence that a packet was lost

What the slide says (Seek W6 L1, slides 8–9)

Example: suppose cwnd and inflight (before triple dup ACK) were both 8 MSS. After triple dup ACK, reduce inflight to 4 MSS. Assume 3 of those 8 MSS are no longer in flight; set **cwnd = 7 MSS** (= 4 + 3). (2) The seq# from the dup ACKs is **immediately retransmitted** — don't wait for an RTO if there is sufficiently strong evidence that a packet was lost.

Common pitfall: Check the arithmetic

Careful with the slide's arithmetic: $7 = 8/2 + 3$, not $8 - 1$. And ssthresh = 4, not 7.

1.5 TCP fast recovery (RFC 2581)

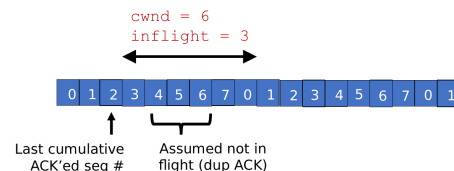
From the lecture slides: Seek W6 L1, slides 10–11 — keep the reduced inflight until a new ACK; $cwnd = 6$, $inflight = 3$

TCP fast recovery (RFC 2581)

- Sender keeps the reduced **inflight** until a **new ACK** arrives
 - New ACK: an ACK for the seq# that was just retransmitted
 - May also include the (three or more) pieces of data that were subsequently delivered to generate the duplicate ACKs
- **Conserve packets in flight**: transmit *some* data over lossy periods (rather than no data, which would happen if $cwnd := 1$)

TCP fast recovery (RFC 2581)

- Keep incrementing $cwnd$ by 1 MSS for each dup ACK



What the slide says (Seek W6 L1, slide 10)

The sender keeps the reduced inflight until a **new ACK** arrives. New ACK: an ACK for the seq# that was just retransmitted (it may also include the three or more pieces of data that were subsequently delivered to generate the duplicate ACKs). **Conserve packets in flight**: transmit *some* data over lossy periods (rather than no data, which would happen if $cwnd := 1$).

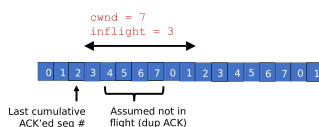
What the slide says (Seek W6 L1, slides 11–14)

Keep incrementing $cwnd$ by 1 MSS for each dup ACK ($cwnd = 6, 7, 8$ with $inflight = 3$ throughout). Eventually a new ACK arrives, acknowledging the retransmitted data and all data in between. **Deflate $cwnd$ to half of $cwnd$ before fast retransmit**: $cwnd$ and $inflight$ are aligned and equal once again ($cwnd = 3$, $inflight = 3$). **Perform additive increase from this point!**

From the lecture slides: Seek W6 L1, slides 12–14 — $cwnd$ inflates to 7, 8 with each dup ACK; on the new ACK it deflates to 3

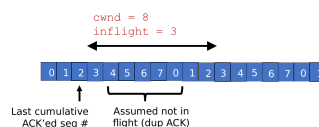
TCP fast recovery (RFC 2581)

- Keep incrementing $cwnd$ by 1 MSS for each dup ACK



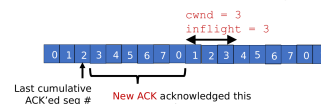
TCP fast recovery (RFC 2581)

- Keep incrementing $cwnd$ by 1 MSS for each dup ACK



TCP fast recovery (RFC 2581)

- Eventually a **new ACK** arrives, acknowledging the retransmitted data and all data in between
- **Deflate $cwnd$ to half of $cwnd$ before fast retransmit**.
 - $cwnd$ and $inflight$ are aligned and equal once again
- Perform **additive increase** from this point!



Formula: Fast recovery

- **Each additional dup ACK**: $cwnd \leftarrow cwnd + 1 \text{ MSS}$ (“window inflation”). Another segment has left the network, so one new segment may be sent: $inflight$ stays at $F/2$ (packet conservation).
- **New ACK** (covers the retransmission): $cwnd \leftarrow ssthresh = F/2$ (“deflation”); now $cwnd = inflight$. Continue with **additive increase** (congestion avoidance) — not slow start.

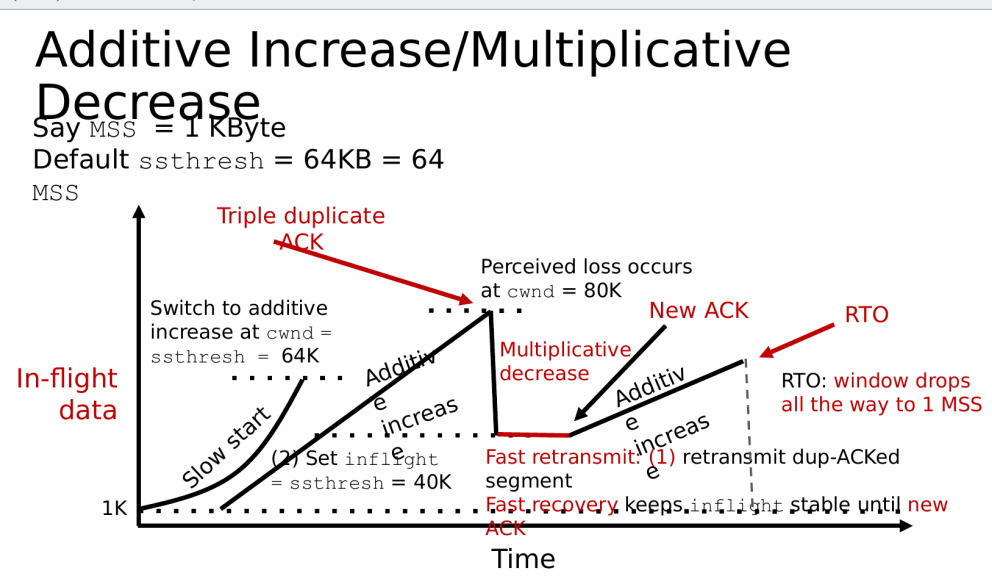
Event (slide numbers, $F = 6$)	cwnd	inflight	ssthresh
Before loss	6	6	—
3rd dup ACK: fast retransmit	$3 + 3 = 6$	3	3
4th dup ACK	7	3	3
5th dup ACK	8	3	3
New ACK: deflate	3	3	3
Then additive increase	4, 5, ... per RTT		

Intuition: Why inflate, then deflate?

Because of the hole, the cumulative ACK is stuck; if cwnd stayed fixed, the sender would soon be unable to send anything (the window's left edge can't move). Inflating by 1 MSS per dup ACK moves the *right* edge instead, letting one new segment out for each one that leaves — ACK clocking continues through the loss. When the retransmission is finally ACKed, the left edge jumps forward past all the buffered data, so the inflation is no longer needed and cwnd is reset to the honest value $F/2$.

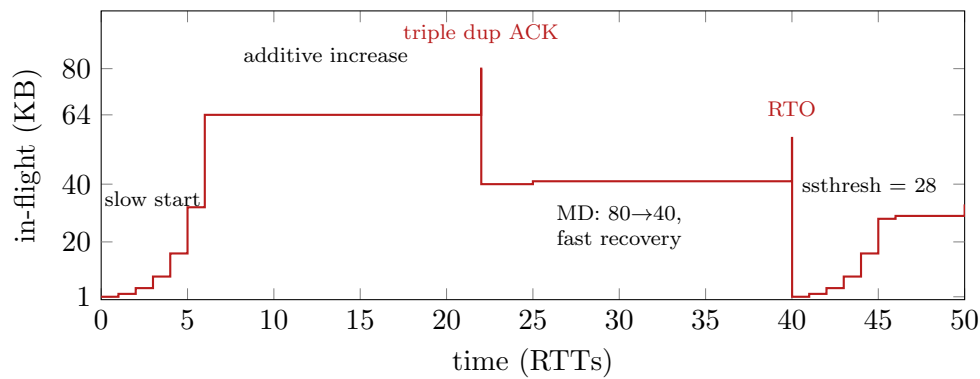
1.6 The full AIMD picture

From the lecture slides: Seek W6 L1, slide 15 — slow start, additive increase, triple dup ACK (MD), fast recovery, RTO



What the slide says (Seek W6 L1, slide 15)

MSS = 1 KB, default ssthresh = 64 KB = 64 MSS. Slow start from 1 K; switch to additive increase at cwnd = ssthresh = 64 K. Perceived loss (triple duplicate ACK) occurs at cwnd = 80 K. Fast retransmit: (1) retransmit the dup-ACKed segment, (2) set inflight = ssthresh = 40 K (multiplicative decrease). Fast recovery keeps inflight stable until the new ACK. Then additive increase again. On an RTO the window drops all the way to 1 MSS.



	Triple duplicate ACK	RTO (timeout)
What it means	one packet lost, later ones delivered: mild congestion	nothing coming back: severe congestion or path failure
sssthresh	$\text{inflight}/2$	$\max(2 \text{ MSS}, \text{cwnd}/2)$
cwnd	$\text{inflight}/2 + 3 \text{ MSS}$, then inflated, then deflated to sssthresh	1 MSS
Next phase	additive increase (congestion avoidance)	slow start up to sssthresh
Retransmit	immediately (fast retransmit)	after the timer expires

2 Module 6.2 — AIMD and the Loss-Reaction Summary

From the lecture slides: Seek W6 L2 — AIMD statement; summary of TCP loss detection & reaction

TCP New Reno performs additive increase and multiplicative decrease of congestion window.

In short, we often refer to this as **AIMD**.

Multiplicative decrease is a part of all TCP algorithms, including BBR.

[It is necessary for **fairness** across TCP flows.]

Summary: TCP loss detection & reaction

- Don't wait for an RTO and then set the `cwnd` to 1 MSS
 - Tantalizing to waiting to get super close to the car in front and then jamming the brakes really hard
- Instead, react proportionately by sensing pkt loss in advance
 - **Fast Retransmit**: **Triple dup ACK**: sufficiently strong signal that network has dropped data, before RTO
 - Immediately retransmit data
 - Multiplicatively decrease in-flight data to **half** of its value
- **Fast Recovery**:
 - Maintain this reduced amount of in-flight data as long as dup ACKs arrive
 - Data is successfully getting delivered
 - When **new ACK** arrives, do **additive increase** from there on

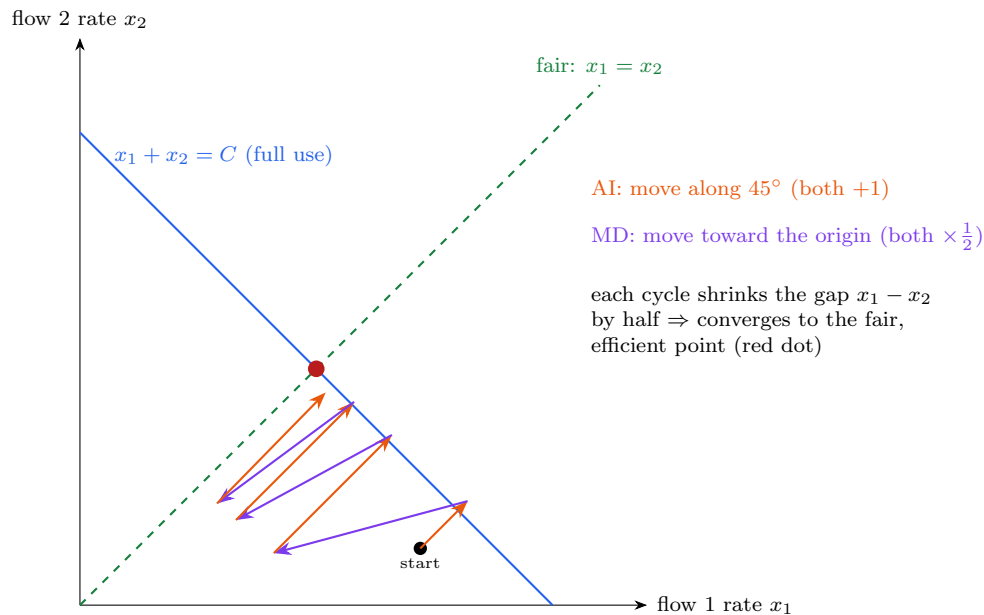
What the slide says (Seek W6 L2, slide 1)

TCP New Reno performs **additive increase and multiplicative decrease** of the congestion window. In short, we often refer to this as **AIMD**. Multiplicative decrease is a part of *all* TCP algorithms, including BBR. [It is necessary for **fairness** across TCP flows.]

Definition: AIMD

Additive Increase, Multiplicative Decrease: grow the window by a constant (1 MSS) per RTT while things are fine; cut it by a constant *factor* (to half) on a loss signal.

2.1 Why multiplicative decrease gives fairness



Intuition: The Chiu–Jain argument (beyond the slides, but it *is* the slide’s bracketed claim)

Two flows share capacity C . Additive increase adds the same amount to both: the *difference* $x_1 - x_2$ is unchanged. Multiplicative decrease halves both: the difference is *halved*. After k loss events the unfairness is $(x_1 - x_2)/2^k \rightarrow 0$. Additive decrease would keep the difference constant forever; multiplicative increase would amplify it. Only AIMD converges to fair and efficient.

Formula: AIMD sawtooth: average throughput

In steady state the window oscillates linearly between $W/2$ and W (loss at W). Average window $= \frac{3}{4}W$, so

$$\overline{\text{Throughput}} \approx \frac{0.75 W \cdot \text{MSS}}{\text{RTT}} \quad \left[\frac{\text{bytes}}{\text{s}} \right]$$

Period of one sawtooth $= W/2$ RTTs (it climbs 1 MSS per RTT from $W/2$ to W).

2.2 Summary: TCP loss detection & reaction

What the slide says (Seek W6 L2, slide 2)

Don’t wait for an RTO and then set `cwnd` to 1 MSS — tantamount to waiting to get super close to the car in front and then jamming the brakes really hard. Instead, **react proportionately by sensing packet loss in advance**. **Fast retransmit:** triple dup ACK is a sufficiently strong signal that the network has dropped data, before RTO. Immediately retransmit data. Multiplicatively decrease in-flight data to half of its value. **Fast recovery:** maintain this reduced amount of in-flight data as long as dup ACKs arrive (data is successfully getting delivered). When a new ACK arrives, do additive increase from there on.

3 Module 6.3 — TCP BBR and Getting to Steady State

3.1 The zoo, and two algorithms compared

From the lecture slides: Seek W6 L3, slide 2 — TCP New Reno vs TCP BBR

Two congestion control algorithms

TCP New Reno

- The most studied, classic “textbook” TCP algorithm
- The primary knob is **congestion window**
- The primary signal is **packet loss (RTO)**
- Adjustment using **additive increase**

TCP BBR

- Recent algorithm developed & deployed by Google
- The primary knob is **sending rate**
- The primary signal is **rate of incoming ACKs**
- Adjustment using **gain cycling** and **filters**

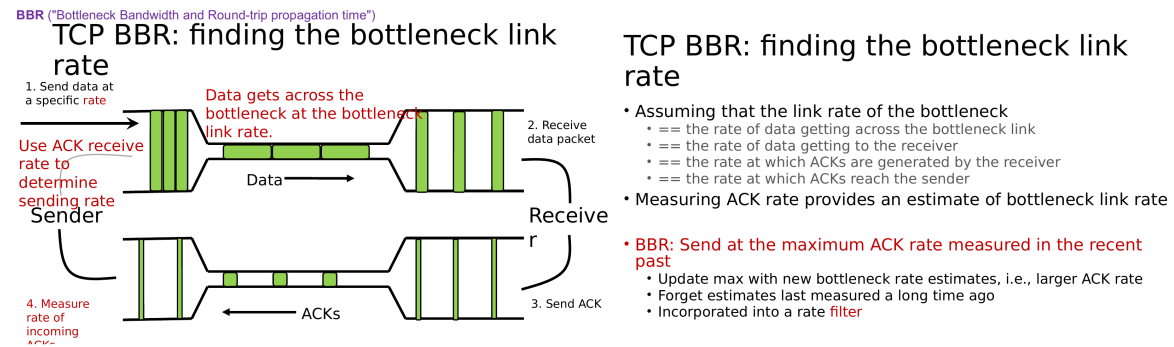
	TCP New Reno	TCP BBR
Origin	the most studied, classic “textbook” algorithm	recent, developed & deployed by Google
Primary knob	congestion window	sending rate
Primary signal	packet loss (RTO, dup ACKs)	rate of incoming ACKs
Adjustment	additive increase (+ MD on loss)	gain cycling and filters
Steady state	sawtooth: fills buffers until loss	tries to sit at the knee: full pipe, empty queue

Intuition: Why a new algorithm?

Loss-based TCP only learns the pipe is full *after* it has filled the bottleneck buffer and overflowed it. With today’s large buffers this means long queues (“bufferbloat”, high RTT) before any signal arrives; and on lossy wireless links, random losses are misread as congestion. BBR asks a different question: “what is the bottleneck rate?” — and measures it directly from ACKs.

3.2 BBR: finding the bottleneck link rate

From the lecture slides: Seek W6 L3, slides 3–4 — send at a rate, measure the ACK rate, send at the max recent ACK rate



What the slide says (Seek W6 L3, slides 3–4)

BBR = “Bottleneck Bandwidth and Round-trip propagation time”. (1) Send data at a specific rate. Data gets across the bottleneck at the bottleneck link rate. (2) The receiver receives data packets; (3) sends ACKs; (4) the sender measures the rate of incoming ACKs and uses the ACK receive rate to determine its sending rate. Assuming that the link rate of the bottleneck == the rate of data getting across the bottleneck link == the rate of data getting to the receiver == the rate at which ACKs are generated by the receiver == the rate at which ACKs reach the sender. Measuring ACK rate provides an estimate of the bottleneck link rate. **BBR: send at the maximum ACK rate measured in the recent past.** Update the max with new bottleneck rate estimates (i.e. larger ACK rate); forget estimates last measured a long time ago; incorporated into a rate filter.

Formula: Bottleneck bandwidth estimate

$$\hat{R}_{\text{btl}} = \max_{\text{recent window}} \frac{\text{data ACKed in an interval}}{\text{interval length}} \quad \left[\frac{\text{bits}}{\text{s}} \right]$$

E.g. 1000-byte segments ACKed every 0.8 ms $\Rightarrow 8000/0.0008 = 10$ Mbps.

Intuition: Why the maximum, and why forget?

The ACK rate can only *underestimate* the bottleneck (if you send slower than it, ACKs come back slower too; noise and cross-traffic also slow them). It can never be faster than the bottleneck. So the max over recent samples is the best estimate. But paths change (route change, other flows leave or arrive); a max remembered from long ago may no longer be true, so old samples expire from the filter (a windowed max, roughly the last 10 RTTs in real BBR).

Beyond the slides: The “RTprop” half of BBR

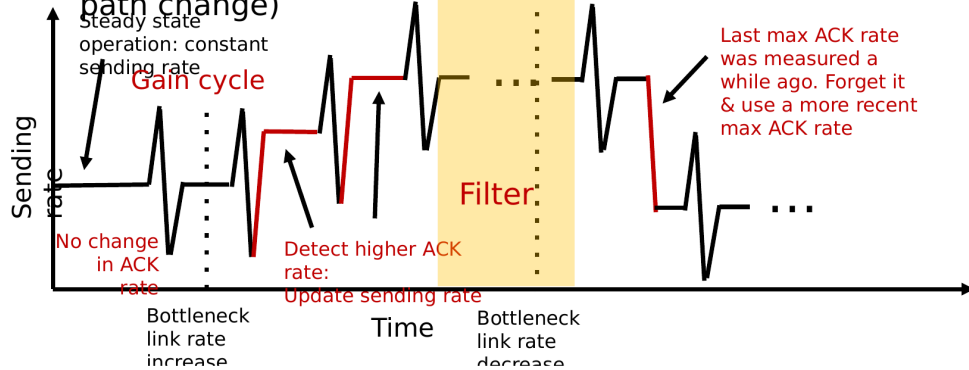
BBR also tracks the minimum RTT seen recently (round-trip propagation time, with no queue). Its target in-flight data is $\hat{R}_{\text{btl}} \times \text{RTT}_{\text{min}}$ — the BDP, i.e. the ideal steady state from Week 5: full pipe, empty queue.

3.3 BBR: adjustments by gain cycling

From the lecture slides: Seek W6 L3, slide 5 — steady rate, periodic gain probes, filter keeps recent max

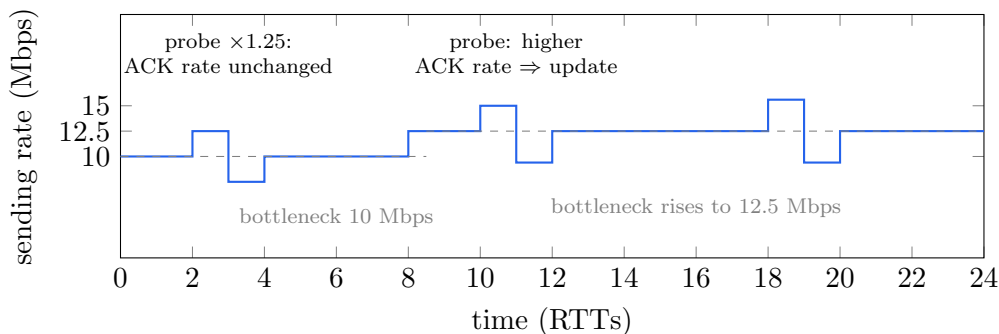
TCP BBR: Adjustments by gain cycling

- BBR periodically increases its sending rate by a gain factor to see if the link rate has increased (e.g., due to a path change)



What the slide says (Seek W6 L3, slide 5)

BBR periodically increases its sending rate by a **gain factor** to see if the link rate has increased (e.g. due to a path change). Steady-state operation: constant sending rate. Gain cycle: no change in ACK rate $\Rightarrow$ go back. Detect higher ACK rate $\Rightarrow$ update sending rate. When the last max ACK rate was measured a while ago, forget it and use a more recent max ACK rate (bottleneck link rate decrease).



Intuition

Gain cycling is BBR's version of "probing". Most of the time it sends at the estimated rate. Occasionally it sends 25% faster for one RTT (gain 1.25): if the ACK rate rises, the pipe had spare capacity, so the estimate goes up; if it doesn't, the extra data only queued, so BBR then sends slightly *below* the rate (gain 0.75) for one RTT to drain that queue, and returns to cruising. Cycle gains in real BBR: [1.25, 0.75, 1, 1, 1, 1, 1, 1].

3.4 Summary: getting to steady state

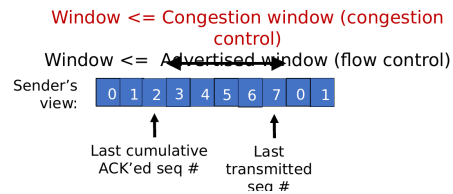
From the lecture slides: Seek W6 L3, slides 6–7 — summary; interaction between flow and congestion control

Summary: Getting to Steady State

- Want to get to highest sending rate that doesn't congest the bottleneck link
- **Slow start**: Exponential increase towards a reasonable estimate of link rate
- **Congestion avoidance**: milder adjustments to get close to correct link rate estimate.
- TCP New Reno: **additive increase**
- TCP BBR: **gain cycling** and filters

Interaction b/w flow & congestion control

- Use $\text{window} = \min(\text{congestion window, receiver advertised window})$
- Overwhelm neither the receiver nor network links & routers



Summary: Getting to steady state (slide)

Want to get to the highest sending rate that doesn't congest the bottleneck link. **Slow start**: exponential increase towards a reasonable estimate of the link rate. **Congestion avoidance**: milder adjustments to get close to the correct link-rate estimate — **TCP New Reno**: additive increase; **TCP BBR**: gain cycling and filters.

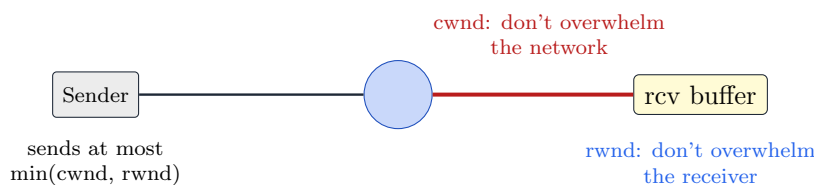
3.5 Interaction between flow control and congestion control

What the slide says (Seek W6 L3, slide 7)

Use $\text{window} = \min(\text{congestion window, receiver advertised window})$. Overwhelm neither the receiver nor network links & routers. $\text{Window} \leq \text{congestion window}$ (congestion control); $\text{Window} \leq \text{advertised window}$ (flow control).

Formula: The effective window

$$W_{\text{eff}} = \min(\text{cwnd}, \text{rwnd}), \quad \text{Throughput} \approx \frac{\min(\text{cwnd}, \text{rwnd})}{\text{RTT}}$$



Intuition

Two independent limits, one from each possible bottleneck. Whichever is smaller decides: on a fast LAN with a slow app, rwnd binds (flow control); on a congested wide-area path with a fast receiver, cwnd binds (congestion control).

4 Formula Sheet

Loss signal (fast)	3 duplicate ACKs (same ACK #)
Fast retransmit ($F = \text{inflight}$ before)	$\text{ssthresh} = F/2$; $\text{inflight} \rightarrow F/2$; $\text{cwnd} = F/2 + 3 \text{ MSS}$; retransmit now
Fast recovery	each extra dup ACK: $\text{cwnd} + 1 \text{ MSS}$; new ACK: $\text{cwnd} = \text{ssthresh}$, then additive increase
On RTO	$\text{ssthresh} = \max(2 \text{ MSS}, \text{cwnd}/2)$; $\text{cwnd} = 1 \text{ MSS}$; slow start
AIMD sawtooth	window between $W/2$ and W ; average $0.75W$; throughput $\approx 0.75 W \cdot \text{MSS}/\text{RTT}$
Sawtooth period	$W/2 \text{ RTTs}$
BBR bandwidth estimate	max recent ACK rate (bytes ACKed / time)
BBR gain cycling	probe at $1.25\times$, drain at $0.75\times$, else $1\times$
Effective window	$\min(\text{cwnd}, \text{rwnd})$

5 Numerical Problems with Worked Solutions

Problem 1. Duplicate ACKs

A sender transmits segments 1–8 (one MSS each, ACK = next expected segment). Segment 3 is lost; all others arrive in order. List the ACKs. When does fast retransmit happen?

Solution

After 1: ACK 2. After 2: ACK 3. After 4, 5, 6, 7, 8: ACK 3 each (5 duplicates). Fast retransmit of segment 3 is triggered by the **third duplicate**, i.e. when segment 6's ACK arrives. When the retransmitted 3 arrives, the receiver (which buffered 4–8) sends **ACK 9**.

Problem 2. Fast retransmit — the slide's example

$\text{cwnd} = \text{inflight} = 8 \text{ MSS}$ when a triple duplicate ACK arrives. Find the new ssthresh , inflight and cwnd .

Solution

$\text{ssthresh} = 8/2 = 4$; $\text{inflight} \rightarrow 4$; $\text{cwnd} = 4 + 3 = 7 \text{ MSS}$. The missing segment is retransmitted at once.

Problem 3. Fast recovery trace

$\text{cwnd} = \text{inflight} = 20 \text{ MSS}$. A triple dup ACK arrives, then 4 more dup ACKs, then a new ACK. Give cwnd , inflight and ssthresh after each event, and cwnd for the next 3 RTTs.

Solution

Event	cwnd	inflight	ssthresh
3rd dup ACK	$10 + 3 = 13$	10	10
4th, 5th, 6th, 7th dup ACK	14, 15, 16, 17	10	10
New ACK	10	10	10
Next 3 RTTs (AI)	11, 12, 13		

Problem 4. *The slide's six-segment example*

Reproduce the slide: before loss $\text{cwnd} = \text{inflight} = 6$. Show the numbers through two further dup ACKs and the new ACK.

Solution

3rd dup ACK: $\text{ssthresh} = 3$, $\text{inflight} = 3$, $\text{cwnd} = 3 + 3 = 6$. Next dup ACK: $\text{cwnd} = 7$. Next: $\text{cwnd} = 8$ (inflight stays 3). New ACK: cwnd deflates to $6/2 = 3 = \text{inflight}$; additive increase from 3.

Problem 5. *RTO vs fast retransmit*

$\text{cwnd} = 32 \text{ MSS}$, $\text{RTT} = 100 \text{ ms}$. Compare how long it takes to get back to a window of 16 MSS after (a) an RTO, (b) a triple dup ACK.

Solution

(a) RTO: $\text{ssthresh} = 16$, $\text{cwnd} = 1$: slow start 1, 2, 4, 8, 16 $\Rightarrow$ **4 RTTs = 400 ms** (plus the RTO wait itself, often $\geq 200 \text{ ms}$). (b) Triple dup ACK: cwnd is 16 as soon as recovery ends, about **1 RTT** later. Fast retransmit/recovery avoids both the RTO wait and the slow-start climb.

Problem 6. *The slide's AIMD figure*

$\text{MSS} = 1 \text{ KB}$, default $\text{ssthresh} = 64 \text{ KB}$. Slow start from 1 KB; switch at 64 K; triple dup ACK at 80 K. (a) RTTs of slow start? (b) RTTs of additive increase before the loss? (c) New ssthresh and in-flight? (d) A later RTO occurs at 56 K: new ssthresh and cwnd ?

Solution

(a) $1 \rightarrow 64$: $2^6 = 64 \Rightarrow$ **6 RTTs**. (b) $80 - 64 =$ **16 RTTs**. (c) $\text{ssthresh} = \text{inflight} =$ **40 K**. (d) $\text{ssthresh} = \max(2, 56/2) =$ **28 K**; $\text{cwnd} = 1 \text{ MSS}$, slow start to 28 K.

Problem 7. *AIMD average throughput*

In steady state a New Reno flow loses a packet (triple dup ACK) whenever its window reaches $W = 80 \text{ MSS}$. $\text{MSS} = 1000 \text{ bytes}$, $\text{RTT} = 100 \text{ ms}$. Find the average throughput and the sawtooth period.

Solution

Window cycles $40 \rightarrow 80$. Average = 60 MSS. Throughput = $60 \times 8000/0.1 =$ **4.8 Mbps**. Period = $W/2 = 40 \text{ RTTs} =$ **4 s**.

Problem 8. *AIMD fairness*

Two flows share a bottleneck of 100 packets/RTT. They start at $x_1 = 70$, $x_2 = 10$. Each RTT both add 1; whenever $x_1 + x_2 > 100$ both halve. Track the difference $x_1 - x_2$ over the first three loss events.

Solution

Sum starts at 80 and grows by 2 per RTT (both add 1).

1. After 11 RTTs: (81, 21), sum $102 > 100 \Rightarrow$ halve: (40.5, 10.5). Difference 60 $\rightarrow$ **30**.
2. Sum 51; after 25 RTTs: (65.5, 35.5), sum 101 $\Rightarrow$ halve: (32.75, 17.75). Difference **15**.
3. Sum 50.5; after 25 RTTs: (57.75, 42.75), sum 100.5 $\Rightarrow$ halve: (28.875, 21.375). Difference **7.5**.

AI never changes the difference; each MD halves it $\Rightarrow$ the flows converge to 50/50.

Problem 9. *BBR bandwidth estimate*

BBR sees 1500-byte segments ACKed at intervals of 1.2 ms. (a) Estimated bottleneck rate? (b) Probe rate with gain 1.25? (c) During the probe, ACKs still arrive every 1.2 ms. What does BBR conclude? (d) Min RTT is 40 ms: target in-flight?

Solution

(a) $12,000/0.0012 = \mathbf{10\text{ Mbps}}$. (b) **12.5 Mbps**. (c) The bottleneck is still 10 Mbps; the extra went into a queue $\Rightarrow$ drain phase (0.75 $\times$) then back to 10 Mbps. (d) $10^7 \times 0.04 = 4 \times 10^5$ bits = **50 KB** $\approx$ 33 segments.

Problem 10. *Effective window*

cwnd = 40 KB, rwnd = 25 KB, RTT = 50 ms. Which control binds, and what is the throughput? What if the app speeds up and rwnd becomes 64 KB?

Solution

$\min(40, 25) = 25$ KB: **flow control** binds. $25,000 \times 8/0.05 = \mathbf{4\text{ Mbps}}$. With rwnd 64 KB: $\min = 40$ KB, **congestion control** binds: $40,000 \times 8/0.05 = \mathbf{6.4\text{ Mbps}}$.

Problem 11. *How many segments sent during fast recovery?*

cwnd = inflight = 12 before a triple dup ACK; then 6 more dup ACKs arrive before the new ACK. How many new segments can the sender send during fast recovery?

Solution

Use the RFC 2581 rule behind the slides: the sender may transmit a new segment whenever cwnd exceeds the amount of outstanding data (the sequence range from the last cumulative ACK), which is still $F = 12$ segments during recovery. After fast retransmit: ssthresh = 6, cwnd = $6 + 3 = 9 < 12$: nothing new can be sent. The 6 further dup ACKs inflate cwnd to 10, 11, 12 (still < 12 , nothing sent), then 13, 14, 15 — each of these last three lets one new segment out. Answer: **3** new segments. Check with the slide's view: halving means only half the window's worth of new data is clocked out while recovering — the first dup ACKs just “pay back” the reduction.

6 Multiple-Choice Questions with Explanations

Q1. Before fast retransmit, TCP's only loss detection was:

- (a) Triple dup ACK
- (b) RTO timer expiration
- (c) ECN
- (d) NAK

(b).

Q2. Duplicate ACKs are ACKs that:

- (a) Arrive twice due to a bug
- (b) Carry the same cumulative ACK number as a previous ACK
- (c) Acknowledge the SYN
- (d) Carry SACK blocks

(b).

Q3. Duplicate ACKs can be caused by:

- (a) Only congestion
- (b) Reordering or a single loss within the window
- (c) Only RTO
- (d) Checksum errors at the sender

(b).

Q4. The default fast-retransmit threshold is:

- (a) 1 dup ACK
- (b) 2 dup ACKs
- (c) 3 dup ACKs
- (d) RTO

(c).

Q5. Why not retransmit after 1 dup ACK?

- (a) It's not allowed
- (b) Mild reordering would cause needless retransmissions
- (c) The RTO is faster
- (d) ACKs have no numbers

(b).

Q6. During fast recovery, “inflight” differs from “cwnd” because:

- (a) cwnd includes segments known to have left the network via dup ACKs
- (b) inflight includes the receiver buffer
- (c) cwnd is in bits
- (d) They never differ

(a).

Q7. On a triple dup ACK with inflight 8 MSS, New Reno sets cwnd to:

- (a) 1
- (b) 4
- (c) 7
- (d) 8

(c). $8/2 + 3$.

Q8. On a triple dup ACK, ssthresh becomes:

- (a) 1 MSS
- (b) $\text{inflight}/2$
- (c) $\text{cwnd} + 3$
- (d) 64 KB

(b).

Q9. Fast retransmit means:

- (a) Retransmit the whole window
- (b) Retransmit the dup-ACKed segment without waiting for RTO
- (c) Double cwnd
- (d) Reset the connection

(b).

Q10. During fast recovery, each additional dup ACK:

- (a) Halves cwnd
- (b) Increases cwnd by 1 MSS
- (c) Sets $\text{cwnd} = 1$
- (d) Triggers another retransmission

(b).

Q11. A “new ACK” in fast recovery is:

- (a) Any ACK
- (b) An ACK covering the retransmitted segment
- (c) A SYN-ACK
- (d) A duplicate

(b).

Q12. When the new ACK arrives, cwnd is:

- (a) Set to 1
- (b) Deflated to half of cwnd before fast retransmit ($= \text{ssthresh}$)
- (c) Doubled
- (d) Left inflated

(b).

Q13. After fast recovery, TCP New Reno continues with:

- (a) Slow start
- (b) Additive increase
- (c) Stop-and-wait
- (d) BBR

(b).

Q14. After an RTO, TCP New Reno continues with:

- (a) Additive increase from $\text{cwnd}/2$
- (b) Slow start from 1 MSS
- (c) Fast recovery

(d) Nothing

(b).

Q15. The purpose of fast recovery is to:

- (a) Send no data during loss
- (b) Keep sending some data (conserve packets) during lossy periods
- (c) Increase RTO
- (d) Close the connection

(b).

Q16. AIMD stands for:

- (a) Adaptive Increase, Multiple Detection
- (b) Additive Increase, Multiplicative Decrease
- (c) Average In-flight, Maximum Delay
- (d) ACK-Induced Multiplicative Drop

(b).

Q17. Multiplicative decrease is necessary for:

- (a) Speed
- (b) Fairness across flows
- (c) Encryption
- (d) Routing

(b).

Q18. Under AIMD, the rate difference between two competing flows:

- (a) Grows each cycle
- (b) Is halved at each multiplicative decrease
- (c) Stays constant
- (d) Doubles on increase

(b).

Q19. The average of an AIMD sawtooth between $W/2$ and W is:

- (a) $W/2$
- (b) $0.75W$
- (c) W
- (d) $2W$

(b).

Q20. TCP BBR's primary knob is:

- (a) Congestion window
- (b) Sending rate
- (c) Advertised window
- (d) TTL

(b).

Q21. TCP BBR's primary signal is:

- (a) Packet loss
- (b) Rate of incoming ACKs
- (c) ECN only
- (d) Receiver window

(b).

Q22. BBR estimates the bottleneck rate as:

- (a) Minimum ACK rate ever seen
- (b) Maximum ACK rate in the recent past
- (c) Average of all ACK rates
- (d) The link rate of the first hop

(b).

Q23. Why does BBR forget old maxima?

- (a) To save memory
- (b) The path may have changed; old estimates may be stale
- (c) RFC requires it
- (d) To be unfair

(b).

Q24. BBR's gain cycling is used to:

- (a) Detect if the link rate has increased
- (b) Encrypt data
- (c) Compute checksums
- (d) Set the TTL

(a).

Q25. New Reno's primary knob and signal are:

- (a) Rate and ACK rate
- (b) Congestion window and packet loss
- (c) TTL and RTT
- (d) rwnd and ECN

(b).

Q26. The sender's effective window is:

- (a) cwnd
- (b) rwnd
- (c) $\min(\text{cwnd}, \text{rwnd})$
- (d) $\max(\text{cwnd}, \text{rwnd})$

(c).

Q27. If $\text{cwnd} = 50$ KB and $\text{rwnd} = 20$ KB, the limit comes from:

- (a) Congestion control
- (b) Flow control

- (c) Both equally
- (d) Neither

(b).

Q28. Slow start in the summary is described as:

- (a) Linear growth
- (b) Exponential increase towards an estimate of link rate
- (c) Constant rate
- (d) Multiplicative decrease

(b).

Q29. In the slide's AIMD figure, a triple dup ACK at 80 K sets in-flight to:

- (a) 1 K
- (b) 40 K
- (c) 64 K
- (d) 80 K

(b).

Q30. The driving analogy for RTO-based reaction is:

- (a) Cruise control
- (b) Getting very close to the car in front, then braking hard
- (c) Changing lanes
- (d) Parking

(b).